

Entwicklungsprojekt 2021/22

von Vincent Luciano und Luzie Ewert



Inhaltsverzeichnis

Alleinstellungsmerkmal.....	Folie 3
Zielhierarchie.....	Folie 4
Risikoanalyse.....	Folie 5
Architekturdiagramm.....	Folie 6
PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 7
PoC: Wie sich die Personen in der Bahn/ im Bus finden.....	Folie 8
PoC: Wie endet eine Fahrt.....	Folie 9
PoC: Ticketgeber dazu bringen, die App zu nutzen.....	Folie 10
Projektplan.....	Folie 11



- **Einmalig, gibt es so noch nicht auf dem Markt**
- **Bietet soziale Interaktion im Alltag**
- **unterstützt soziale Bereitschaft der Menschen**
- **unterstützt sozial Benachteiligte**
- **stellt Verbindung zwischen Personen mit Ticket und Personen ohne Ticket her, die sich nicht kennen**



Da sich unsere Use Cases, welche wir als PoC's darstellen wollen, noch einmal geändert hatten, wollten wir auch die beziehungsweise das Alleinstellungsmerkmal dementsprechend anpassen. Mit Veränderung der PoC's veränderte sich auch der Fokus unserer Herangehensweise und somit auch das Alleinstellungsmerkmal. Schwerpunkt war es daher nun, die beiden Personen miteinander zu verbinden, durch unsere Anwendung und ohne, dass diese sich vorher kannten. Da es, wie bereits in einem der anderen Punkte aufgegriffen, keine ähnliche Anwendung gibt, die diesen Service anbietet, stellt dies auch unseren Schwerpunkt dar und bildet das Hauptalleinstellungsmerkmal.

Zielhierarchie

Strategische Ziele

- Umwelt schonen
- Weniger Schwarzfahren
- Mehr Kontakt zwischen Menschen
- Jeder Mensch kann fahren
- Dem Nutzer einfache Bedienung ermöglichen
- Weitflächiges Netzwerk

Taktische Ziele

- Menschen dazu bringen die Anwendung zu nutzen
- Eine Community Bauen

Operative Ziele

- Einen Prototyp bauen
- Einen MVP bauen
- Erste Nutzer bekommen und Werbung
- Feedback bekommen
- stetige Updates, zur Verbesserung der Usability



In der Zielhierarchie sollte es vor allem darum gehen, die Alleinstellungsmerkmale möglich zu machen. In der Nachbearbeitung wurden die Punkte dessen nochmal angepasst und es wurde geschaut, in wie fern die als Proof- of- Concept dargestellten Use Cases Bedingungen an die Zielhierarchie stellen und durch welche operativen Ziele man diesen entgegenwirken kann. Da wir in unseren vorherigen PoC's bereits diese Überlegungen angestellt hatten und uns nun auf kleinere Use Cases beschränkt hatten, hatten wir bereits einige Ziele abgedeckt. Um allerdings den PoC's entgegenzukommen, mussten ein paar wenige Aspekte, wie das ausbauen der Usability in der Zielhierarchie nochmals betont werden und wurde dementsprechend und den statischen und operativen Zielen nochmals aufgegriffen.

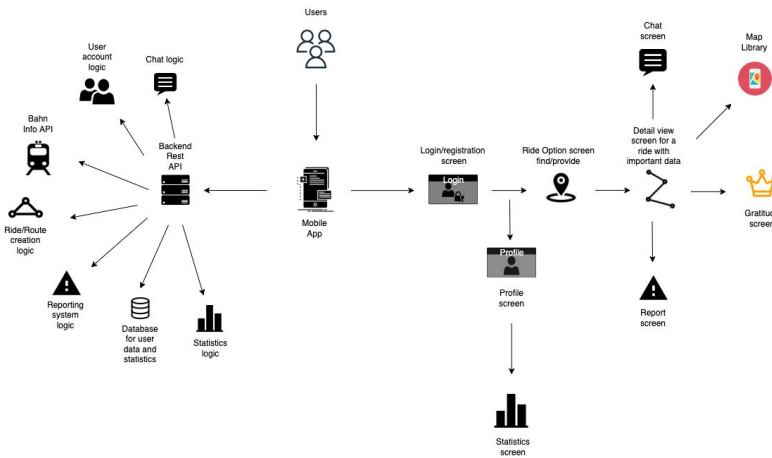
Risikoanalyse

- **Passende API finden**
- **APIs könnten ausfallen**
- **Fehlerhafte Entscheidung der Programmiersprache**
- **Netzwerk kreieren - setzt viele Nutzer voraus**
- **Wir setzen auf die Solidarität der Menschen**
- **Entstehung von spät erkannte Bugs in der Entwicklungsphase**
- **Ausfall einer der Teammitglieder**
- **Rechtliche Konflikte mit den Verkehrsunternehmen**
- **Genug Nutzer erreichen**
- **Große Internetabhängigkeit**



Nachdem die nicht technischen Risiken bereits erörtert wurden und die technischen nun in Form von PoC's dargestellt werden sollten, ging es auch darum sich Gedanken über die Technologien der Anwendung zu machen und welche da am besten für unser Vorhaben geeignet sind. Hierfür wurden zwei neue Punkte der Risikoanalyse hinzugefügt, welche als Risiko abgewägt werden mussten und wo entsprechend dem minimalsten Risiko hingehend gehandelt werden sollte. Diese Aspekte traten unter anderem auch während der Spezifizierung der PoC's auf, da wir dabei bemerkten, dass das Bilden eines Netzwerks und die Suche nach einer anderen Person nicht ohne eine stabile Internetverbindung möglich sein würde. Dieses Risiko galt es abzuwägen und nach Alternativen zu schauen. Außerdem mussten wir einschätzen, wie sehr eine instabile Internetverbindung die Anwendung einschränken würde und an welchen Stellen man versuchen könnte, auf eine nicht Internet abhängige Alternative umsteigen zu können.

Architekturdiagramm



Das Architekturdiagramm ließ sich aus der vorherigen Arbeit ableiten, in dem in der Risikoanalyse überlegt wurde, für welches Endgerät die Anwendung letztendlich entwickelt werden sollte und wodurch man die meisten Use Case's abdecken kann. Da wir beschlossen eine dynamische Suche möglich zu machen und dies unser meist genutzter Anwendungsfall sein würde, fiel die Wahl relativ schnell von einer Web Anwendung auf eine App umzusteigen. Hierbei werden REST API und User Interface getrennt behandelt, um so viel Datenverkehr einzusparen wie möglich und damit die Nutzer stets bei der Nutzung der App nur auf die Informationen zugreifen müssen die bereits vom System bereitgestellt werden und die Abfragen im Hintergrund laufen können. Was der Nutzer am Ende im UI zu sehen bekommt, sind also nur die Screens, welche die Informationen überbringen. Jegliche API's und unsere Datenbank sollen somit im Backend ausgelagert werden. Hierbei war es uns wichtig, alle nötigen Funktionen, für die Umsetzung der PoC's aufzuführen. Auch Funktionen, wie das Report System wurden im Architekturdiagramm dargestellt, um eine Vorstellung davon zu bekommen, wie die App im besten Falle aussehen würde. Allerdings sind einige Funktionen, wie dem Report System zum Beispiel, nicht

unbedingt im Projekt eingeplant und werden in diesem Umfang nicht umgesetzt werden können, damit wir uns auf die wichtigsten Funktionen konzentrieren können.

PoC: Jemanden finden, der die gleiche Strecke fährt

Exit-Kriterien:

- Daten der Nutzer sammeln(Aufenthalt, Ziel, etc.)
 - Eingabefeld/ manuell
 - GPS
 - naheliegende ÖPNV anzeigen
 - Haltestellen der ÖPNV aufführen -> Ausstiegspunkt auswählbar
- Daten abgleichen -> bei Übereinstimmung, bildet sich "Match"
- für aktuelle ÖPNV Informationen und Fahrpläne werden API's benutzt(HERE API, DB API, etc.)

Fail-Kriterien:

- schlechte Internetverbindung
- kein Zugriff auf Live- Daten der ÖPNV
- gestörtes GPS Signal

Fallbacks:

- Informationen der ÖPNV manuell als kompaktere Version eingeben
- unvollständige Datensätze können durch Algorithmen angepasst werden
- Live Daten werden durch die Eingabe der Nutzer ersetzt



Als erster Use Case, welchen wir mittels eines Proof- of- Concept's auf die technische Umsetzung prüfen wollen, filterte sich recht schnell unser ebenfalls genanntes Alleinstellungsmerkmal heraus, wie sich eine Person ohne Ticket und eine Person mit Ticket überhaupt finden. Es ging darum, zu überlegen, aus welchen Gründen sich diese zwei Personen zusammen tun würden und wie der Prozess des Findens stattfinden würde. Die beiden Personen hätten somit als Schnittstelle die gemeinsame Richtung in die sie fahren und aus welcher sie kommen. Da wir hier sehr abhängig von den öffentlichen Verkehrsmitteln und deren Linien sowie Abfahrtzeiten sein würden, musste zuerst ein Weg gefunden werden, an diese Informationen zu kommen und diese auch ständig aktualisieren zu können. Da einige Web Services diese Informationen bereits nutzen und auch zur Verfügung stellen, kam die Überlegung auf, eine API zu nutzen, über welche wir ebenfalls die nötigen Informationen bekommen würden.

Nach einigem Abwägen wurde ebenso über den Zeitpunkt entschieden, wann sich die Personen in der Regel nach einer Mitfahrgelegenheit umschauen würden, und inwiefern dies Einfluss auf die Nutzung der Anwendung haben würde. Da eine Bahn- oder Busfahrt meist von

vielen Faktoren abhängig ist und nur selten weit im Voraus geplant wird, entschieden wir uns dazu, eine dynamische Suche möglich zu machen. Das bedeutet, dass der Nutzer in der Lage sein sollte, zu jeglichem Zeitpunkt, nach einer Mitfahrgelegenheit suchen zu können. Um jeden Fall abdecken zu können und wenige Ausfälle geplanter Fahrten zu garantieren, sollte somit der Zeitpunkt der Suche kurz vor einer Fahrt stattfinden können. Dies hat zum Vorteil, dass die beiden Personen mit großer Wahrscheinlichkeit die geplante Bahn oder den geplanten Bus/Zug auch bekommen und es sich als einfacher gestaltet, eine Fahrt kurz vorher zu planen, ohne in Zeitstress zu geraten, auf Grund von eigenen Verzögerungen oder ähnlichem.

Um dem Nutzer so viel Arbeit abzunehmen, wie möglich, sollte auch die Bedienung zum Eingeben der Fahrtinformationen so einfach wie möglich gestaltet sein. Damit der Nutzer also nicht alle Daten manuell eingeben muss und wir die Koordinaten der Stationen ja bereits durch die vorhandenen API's bekommen würden, bietet es sich an, auf die GPS Daten der Nutzer zuzugreifen. Darüber können sowohl die am nächsten liegenden Stationen ermittelt werden, als auch der Standort des Nutzers was für den weiteren Verlauf von Vorteil ist. Da das GPS Signal meist beständig ist in der Stadt bietet es einen sicheren und einfachen Weg.

PoC: Wie sich die Personen in der Bahn/ im Bus finden

Exit-Kriterien:

- Chat Funktion
- Einstiegsbereich Abfrage
 - Eingabefeld/ manuell
 - Einstiegsbereich wird dem Fahrt Suchenden übermittelt
- Alternativ zu Einstiegsbereich Abfrage -> Live Location
 - GPS
 - Live Location Abfrage
 - Location Daten werden auf einer eingebetteten Karte angezeigt

Fail-Kriterien:

- Schlechte Internetverbindung
- Erfolgreiche Implementation der Einstiegsbereich Abfrage kann durch viele verschiedene Faktoren scheitern
 - unterschiedliche räumliche Aufteilungen bei Bahn, Bus, Zug
 - Menschliche Fehler: indem eine Änderung des geplanten Einstiegsbereichs vergessen wird

Fallbacks:

- Explizit die Chat Funktion nutzen



Bei dem Use Case, wie sich zwei Personen, die bereits ein "Match" gebildet haben, da sie die gleiche Strecke fahren, wurde zuerst über eine Möglichkeit nachgedacht den Datenverkehr möglichst klein zu halten. Durch wenige Klicks sollte der Nutzer in der Lage sein, die andere Person finden zu können. Die offensichtlichste Möglichkeit war für uns vorerst einen Chat einzubauen, durch den die Nutzer sich nach bilden eines "Matches" vor Ort finden können, ob in Bus/Bahn oder an der Haltestelle. Allerdings ging es auch zu bedenken, dass einige Anhaltspunkte sich überschneiden und somit durch andere Mechanismen kommuniziert werden könnten. So könnte man eine Angabe, mittels eines Buttons, seitens der Ticket besitzenden Person, vorab geben, in welchen Teil der Bahn man zum Beispiel einsteigen möchte. Zudem kam die Überlegung auf, ob Profilbilder den beiden Personen das Finden ebenfalls vereinfachen würde, da sie so wüssten, nach wem sie suchen müssen.

Ein weiterer Punkt, welcher das Finden deutlich vereinfachen würde, wäre es, wenn die Personen den Standort des jeweils anderen immer einsehen könnten. Mittels GPS Ortung und einer damit zusammenhängenden Kartenansicht könnte man eine solche Funktion

ermöglichen. Da wir bereits zuvor schon überlegt hatten GPS in unserer Anwendung zu integrieren, würde dies keinen neuen Schwierigkeitsfaktor darstellen und könnte nur für diese Zwecke weiterverwendet werden.

PoC: Wie endet eine Fahrt

Exit-Kriterien:

- anhand der Live ÖPNV Daten wird die Fahrt getrackt
 - beim Erreichen des Ausstiegspunkts, wird Fahrt automatisch beendet
- GPS Daten der Nutzer werden benötigt
 - sobald ein "Match" gefunden wurde & das geplante Verkehrsmittel abfährt, aktiviert sich Radius Messer
 - prüft ob Personen etwa 50-100 m voneinander entfernt sind
 - falls nicht, bekommt die Person mit Ticket eine Push- Benachrichtigung, zum Bestätigen der beendeten Fahrt

Fail-Kriterien:

- schlechte Internetverbindung
- schlechtes GPS Signal
- Kein passendes Live ÖPNV Daten API

Fallbacks:

- statischer Button ersetzt GPS Funktion



Da eine Person auf ihr Ticket nur eine weitere Person mitnehmen kann, kann auch nur eine Person mit einer anderen Person auf einmal ein "Match" eingehen. Um eine neue Person annehmen zu können müsste die Person mit Ticket somit in der Lage sein, die Fahrt beenden zu können. Auch andere Situationen in denen sich eine der Personen unwohl in der Situation fühlen könnte und die Fahrt vorzeitig beenden möchte könnten somit abgedeckt werden. Auch hier wollten wir vermeiden, dass der Nutzer ständig auf die Anwendung zugreifen muss und diese so viel automatisch ausführen kann, wie möglich. Wenn man sich die Deutsche Bahn App nun anschaut, sieht man, dass sobald eine Fahrt eingegeben wurde, man auf dem Screen sehen kann, wo die Bahn gerade ist und um welche Uhrzeit sie sich an der gewünschten Ausstiegsstation befinden wird. Durch die API's, die von der Deutschen Bahn zur Verfügung gestellt werden, hatten wir gehofft, diese Informationen zu bekommen und zu nutzen, um, wenn die Person ohne Ticket einen Ausstiegspunkt auswählt, die Fahrt der Bahn oder des Busses so zu tracken, dass, sobald die Bahn oder der Bus diese Station erreicht, die Fahrt automatisch beendet wird.

Um dem Nutzer trotzdem noch die Möglichkeit zu lassen, die Fahrt

vorzeitig zu beenden, oder, falls die API das trocken der Busse und Bahnen nicht ermöglicht, auf eine manuelle Alternative umsteigen zu können, soll es ebenfalls einen Button auf dem Personen mit Ticket UI geben, der dies ermöglicht.

PoC: Ticketgeber dazu bringen, die App zu nutzen

Exit-Kriterien:

- motivierende Screens und Push- Benachrichtigungen
- durch Sammeln der Nutzer Daten werden Statistiken erstellt
- Statistiken, die zeigen
 - wie vielen Nutzern am Tag schon geholfen wurde
 - welche persönlichen Ziele man erreicht hat/ wie vielen Personen man geholfen hat

Fail-Kriterien:

- Datenschutz
 - falls viele Nutzer nicht zustimmen, dass deren Daten benutzt werden können, ist eine erfolgreiche Implementation nur bedingt möglich

Fallbacks:

- ausschließlich Push- Benachrichtigungen versenden



Da wir aus rechtlichen Gründen eine Bezahlung nicht möglich machen können, musste ein neuer Anreiz gefunden werden, weshalb die Personen mit Ticket die Anwendung nutzen wollen. Bei unserer Recherche und einem Brainstorming haben wir uns Gedanken über andere Apps und Web Anwendungen gemacht die dem Nutzer keinen Eigennutzen bieten, jedoch trotzdem tausende von Menschen dazu überzeugen sie zu nutzen. Dabei sind wir unter anderem auf Spenden-Apps wie ShareTheMeal gestoßen. Bei dieser App geht es darum regelmäßig kleine Spenden an Projekte seiner Wahl zu spenden, bei denen kein monatlicher Beitrag abgezogen wird sondern der Nutzer je nach beliebigen die App öffnen und durch einen Klick spenden kann. Hierbei wird durch Statistiken gezeigt wie viele Menschen zum Beispiel bereits bei welchem Projekt geholfen haben. Ein ähnliches Konzept sollte auch bei unserer App den Nutzer dazu bringen und motivieren die App weiterhin zu nutzen und diesem ein gutes Gefühl überbringen. Daher überlegten wir uns ebenfalls mit Statistiken, sofern dies mit dem Datenschutz vereinbar ist und zudem, durch positiven Zuspruch, in Form von Push- Benachrichtigungen oder ganzen Screens, nach Abschliessen einer Fahrt, zu arbeiten.

Projektplan

Meilenstein	Zeitpunkt	Luzie	Vincent
Vorstudie:			
Exposé	11.10.2021	70%	30%
Marktrecherche	08.10.2021	50%	50%
1. Audit			
Projektplan	15.10.2021	40%	60%
Architekturdiagramm	13.10.2021	30%	70%
Domänenmodell	13.10.2021	70%	30%
Alleinstellungsmerkmal	31.10.2021	70%	30%
Proof of concepts	20.10.2021	60%	40%
Erste Risiken	27.10.2021	50%	50%
Zielhierarchie	31.10.2021	30%	70%
Stakeholderanalyse	27.10.2021	50%	50%
Präsentation	03.11.2021	50%	50%

2. Audit			
Proof of concepts iteration	22.11.2021	50%	50%
Architekturdiagramm iteration	20.11.2021	30%	70%
Zielhierarchie iteration	20.11.2021	60%	40%
Risiken iteration	21.11.2021	60%	40%
Projektplan iteration	20.11.2021	30%	70%
Erste Struktur & Code	22.11.2021	40%	60%
Präsentation	1.12.2021	50%	50%
3. Audit			
Erster Prototype	?	?	?
Durchgeführte PoCs	?	?	?
Modelle iteration	?	?	?
Anwendungslogik diagramm	?	?	?

Für den 2. Audit wollten wir unsere Proof- of- Concepts weiter ausarbeiten und technisch erörtern. Außerdem sollten die Artefakte des ersten Audits noch einmal überarbeitet werden und den neuen PoC's entsprechend angepasst werden. Außerdem wurde das Gerüst des Projekts bereits im Code gebaut und ein erster Algorithmus wurde begonnen.